

Unit 5

Implementation and Maintenance

Software Analysis & Design | CSIT 5th

Instructor: Pratiksha Poudel

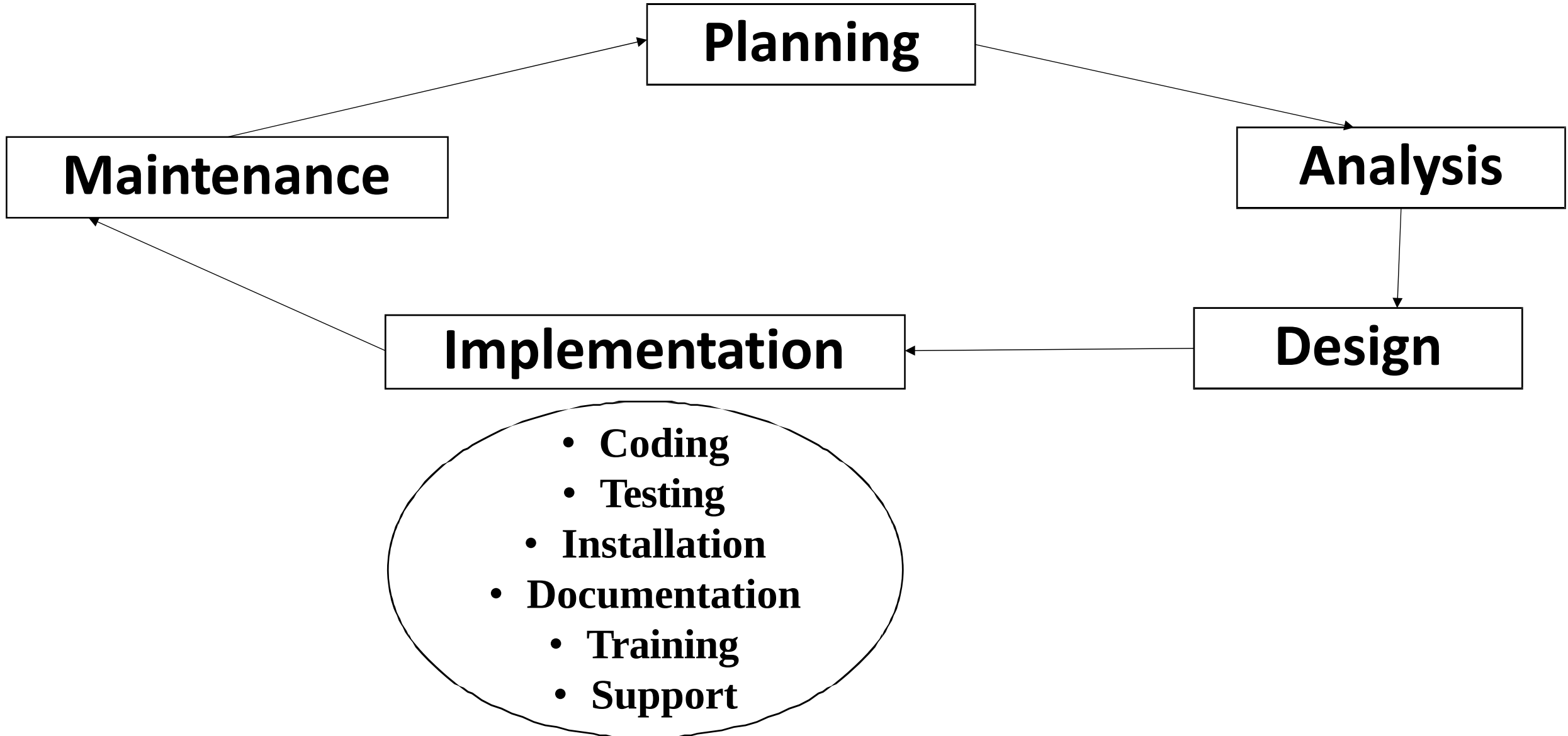
SYSTEM IMPLEMENTATION

- **System Implementation** is the process of converting the physical system specification into working and reliable software .
- Depending on the size and complexity of the system, coding can be involved, intensive activity.
- Once coding has begun, the testing process can begin and proceed in parallel. As each program module is produced, it can be tested individually, then as part of a larger program, and then as part of a larger system.
- Although testing is done during implementation, we must begin planning for testing earlier in the project.
- Planning involves determining what needs to be tested and collecting test data. This is often done during the analysis phase because testing requirements are related to system requirements.

SYSTEM IMPLEMENTATION

- Installation is the process during which the current system is replaced by the new system. This includes conversion of existing data, software, documentation, and work procedures to those consistent with the new system.
- Although the process of documentation proceeds throughout the life cycle, it receives formal attention during the implementation phase because the end of implementation largely marks the end of the analysis team's involvement in systems development.
- Larger organizations also tend to provide training and support to computer users throughout the organization. Some of the training and support is very specific to particular application systems, whereas the rest is general to particular operating systems or off-the-shelf software packages.

Implementation



Software Application Testing

- Testing is the process of executing a program with the intention of finding errors.
- It is the process used to identify the correctness, completeness and quality of developed computer software.
- Software testing can be stated as the process of verifying and validating that a software or application is bug free, meets the technical requirements as guided by its design and development and meets the user requirements effectively and efficiently with handling all the exceptional and boundary cases.
- Software testing is a method of assessing the functionality of a software program. The process of software testing aims not only at finding faults in the existing software but also at finding measures to improve the software in terms of efficiency, accuracy and usability.
- It mainly aims at measuring specifications, functionality and performance of a software program or application.

Software Application Testing

- Software testing is a process that should be done during the development process. In other words software testing is a verification and validation process.

Verification

- **Verification** is the process of checking that a software achieves its goal without any bugs. It is the process to ensure whether the product that is developed is right or not, to make sure the product behaves the way we want it to.
- Verification means **Are we building the product right?**

Validation

- **Validation** is the process to make sure the product satisfies the specified requirements at the end of the development phase.
- to make sure the product is built as per customer requirements.
- Validation means **Have you build the right product?**

Difference between Verification And Validation

➤ <i>Verification</i>	➤ <i>Validation</i>
Verification is to check whether the software conforms to specifications.	Validation is to check whether software meet the customer requirements.
It does not involve executing the code.	It always involves executing the code.
It is human based checking of documents and files.	It is computer based execution of program
Verification is done by development team to provide that the software is as per the specifications	Validation is carried out with the involvement of client and testing team.
It, generally comes first, done before validation.	It generally follows after verification.

Purpose of Software Testing

- Testing verifies that the system meets the different requirements including, functional, performance, reliability, security, usability and so on.
- This verification is done to ensure that we are building the system right.
- It is the process of executing a program/application under positive and negative conditions by manual or automated means. It checks for the :-
 - ❖ Specification
 - ❖ Functionality
 - ❖ Performance

Testing Methodologies

Black box testing

White box testing

Black Box Testing:

- It is defined as a testing technique in which functionality of the Application Under Test is tested without looking at the internal code structure, implementation details and knowledge of internal paths of the software.
- This type of testing is based entirely on software requirements and specifications.
- Black box testing is a high level of testing that focuses on the behavior of the software. It involves testing from an external or end-user perspective. Black box testing can be applied to virtually every level of software testing: system, and acceptance.

White Box Testing:

- White Box Testing is a testing technique that examines the program structure and derives test data from the program logic/code.
- It is a software testing methodology that uses a program's source code to design tests and test cases for quality assurance (QA).
- The code structure is known and understood by the tester in white box testing. White box testing is also called glass box testing because the test cases are totally visible to the general users and they can also make test cases.

Difference between Black Box Testing and White Box Testing are:

Black Box Testing	White Box Testing
It is the software testing method which is used to test the software without knowing the internal structure of code or program.	It is the software testing method in which internal structure is being known to tester who is going to test the software.
This type of testing is carried out by testers, end users.	Generally, this type of testing is carried out by software developers, tester, QA.
Implementation Knowledge is not required to carry out Black Box Testing	Implementation Knowledge is required to carry out White Box Testing
Programming Knowledge is not required to carry out Black Box Testing	Programming Knowledge is required to carry out White Box Testing
Testing is applicable on higher levels of testing like System Testing, Acceptance Testing.	Testing is applicable on lower level of testing like Unit Testing, Integration Testing.

Difference between Black Box Testing and White Box Testing are:

Black Box Testing	White Box Testing
Testing primarily concentrate on the functionality of the system under test.	Testing is primarily concentrated on the testing of program code of the system under test like code structure, branches, conditions, loops etc.
The main aim of this testing is to check on what functionality is performing by the system under test.	The main aim of this testing is to check on how system is performing.
It can be started based on Requirement Specifications documents.	It can be started based on Detail Design documents.
The Functional testing, Behavior testing, Close box testing is carried out under Black Box Testing, so there is no required of the programming knowledge.	The Structural testing, Logic testing, Path testing, Loop testing, Code coverage testing, Open box testing is carried out under White Box Testing, so there is compulsory to know about programming knowledge.

Testing Levels

- Unit testing
- Integration testing
- System testing

UNIT TESTING

- It focuses on the smallest unit of software design. In this, we test an individual unit or group of interrelated units. It is often done by the programmer by using sample input and observing its corresponding outputs.
- It is also called component testing. It is performed on standalone module to check whether it is developed correctly.

The diagram illustrates a login module and its associated unit tests. On the left, there is an orange rounded rectangle labeled "Login". To its right is a "customer login" form with two input fields: "login id" and "password", and a "Login Now" button. Further to the right are three green boxes representing test cases:

- Enter Valid Login ID & Password
- Enter inValid Login ID & Password
- Empty Login ID & Click Login

INTEGRATION TESTING

- The objective is to take unit tested components and build a program structure that has been dictated by design. Integration testing is testing in which a group of components is combined to produce output.
- After integrating the modules, you need to see if the combined modules work together or not. This type of testing is known as integration testing. You need to perform fewer integration tests than unit tests.
- Integrated testing is carried out by testers.

SYSTEM TESTING

- This software is tested such that it works fine for the different operating systems. It is covered under the black box testing technique. In this, we just focus on the required input and output without focusing on internal working.

The system as a whole is tested to uncover requirement errors.

- Verifies that all system elements work properly and that overall system function and performance has been achieved.

Types:

- Alpha Testing
- Beta Testing
- Acceptance Testing
- Performance Testing

- **Alpha Testing** : It is carried out by the test team within the developing organization .
- **Beta Testing** : It is performed by a selected group of friendly customers.
- **Acceptance Testing** :It is performed by the customer to determine whether to accept or reject the delivery of the system.
- **Performance Testing** :It is carried out to check whether the system meets the nonfunctional requirements identified in the SRS document. (Availability, Security, Safety, Reliability, Adaptability are considered)

Difference

Alpha Testing	Beta Testing
Alpha testing involves both the white box and black box testing.	Beta testing commonly uses black box testing.
Alpha testing is performed by testers who are usually internal employees of the organization.	Beta testing is performed by clients who are not part of the organization.
Alpha testing is performed at developer's site.	Beta testing is performed at end-user of the product.
Reliability and security testing are not checked in alpha testing.	Reliability and security are checked during beta testing.
Alpha testing requires a testing environment or a lab.	Beta testing doesn't require a testing environment or lab.
Developers can immediately address the critical issues or fixes in alpha testing.	Most of the issues or feedback collected from beta testing will be implemented in future versions of the product.

Types of Performance Testing:

- Stress Testing
- Volume Testing
- Configuration Testing
- Compatibility Testing
- Regression Testing
- Recovery Testing
- Maintenance Testing
- Documentation Testing
- Usability Testing

Installation

- The process of moving from the current information system to the new one is called installation.
- In this activity, all the users must give up their reliance (dependence/confidence/trust) on the current system and begin to rely on new system.
- There are different ways and approach that an organization decides to use will depend on the scope and complexity of the change associated with the new system and the organization's risk factor. Different ways of installation are:

Installation

- 1. Direct Installation:** Changing over from the old information system to a new one by turning off the old system when the new one is turned on. Direct installation requires a complete installation of the whole system.

This approach is a high risk approach because there may still be major problems that won't be uncovered until the system has been in operation. An errors resulting from the new system will have a direct impact on the users and how they do their jobs is depends on the new system. If the system fails to be used on that specific date, delay may occur until the errors are solved.

Installation

2. Parallel Installation

Running the old information system and the new one at the same time until management decides the old system can be turned off. All of the work done by the old system is concurrently performed by the new system. Outputs are compared to help determine whether the new system is performing as well as the old. Errors discovered in the new system do not cost the organization much, if anything, because errors can be isolated and the business can be supported with the old system.

Because all work is essentially done twice, a parallel installation can be very expensive, running two systems implies employing two staffs to operate and maintain. A parallel approach can also be confusing to users because they must deal with both systems. A parallel approach may not be feasible, especially if the users of the system cannot tolerate redundant effort or if the size of the system is large.

Installation

3. Pilot Installation

It is also known as single-location installation. Rather than converting all of the organization at once, single location installation involves changing from the current to the new system in only one place or in a series of separate sites over time. The single location may be a branch office, a single factory, or one department, and the actual approach used for installation in that location may be any of the other approaches. The key advantage to single location installation is that it limits potential damage and potential cost by limiting the effects to a single site.

Once management has determined that installation has been successful at one location, the new system may be deployed in the rest of the organization, possibly continuing with installation at one location at a time. Problems with the system can be resolved before deployment to other sites. Even though the single location approach may be simpler for users, it still places a large burden on information system staff to support two versions of the system. Problems are isolated at one site at a time. If staff members can devote all of their efforts to success at the pilot site. If different locations require sharing of data, extra programs will need to be written to synchronize the current and new systems.

Installation

4. Phased Installation

It is also called staged installation. Different parts of the old and new systems are used in cooperation until the whole new system is installed. By converting gradually, the organization's risk is spread out over time and place. Also phased installation allows for some benefits from the new system before the whole system is ready.

For example, a new data-capture methods can be used before all reporting modules are ready. For a phased installation, the new and replaced systems must be able to coexist and probably share data. Thus, bridge programs connecting old and new databases and programs often must be built. Sometimes the new and old systems are so incompatible that pieces of the old system cannot be incrementally replaced so this strategy is not feasible. A phased approach requires careful version control, repeated conversations at each phase, and a long period of change, which may be frustrating and confusing to users.

Documenting the System

Documentation is the process of collecting, organizing, storing and maintaining a complete record of system and other documents used or prepared during the different phases of the life cycle of system. System cannot be considered to be complete, until it is properly documented. Proper documentation of system is necessary due to the following reasons:

1. It solves the critical/essential problem of an individual for an organization. Even if the person, who has designed or developed the system, leaves the organization, the documented knowledge remains with the organization, which can be used for the continuity of that software.
2. It makes system easier to modify and maintain in the future. The key to maintenance is proper and dynamic documentation. It is easier to understand the concept of a system from the documented records.
3. It helps in restarting a system development, which was postponed due to some reason. The job need not be started from scratch, and old ideas may still be easily summarized from the available documents, which avoids duplication of work, and saves lot of times and effort.

Types of Documentation

1. **System Documentation:** System documentation records detailed information about a system's design specification, its internal workings and its functionality. System documentation is intended primarily for maintenance programmers. It contains the following information:
 - A description of the system specifying the scope of the problem, the environment in which it functions, its limitation, its input requirement, and the form and type of output required.
 - Detailed diagram of system flowchart and program flowchart.
 - A source listing of all the full details of any modifications made since its development.
 - Specification of all input and output media required for the operation of the system.
 - Problem definition and the objective of developing the program.
 - Output and test report of the program.
 - Upgrade or maintenance history, if modification of the program is made.

Types of Documentation

1. **System Documentation:** There are two types of system documentation. They are:
 - i. **Internal Documentation:** Internal documentation is part of the program source code or is generated at compile time.
 - ii. **External Documentation:** External documentation includes the outcome of structured diagramming techniques such as dataflow and entity-relationship diagrams.
2. **User Documentation:** User documentation consists of written or other visual information about an application system, how it works and how to use it. User documentation is intended primarily for users. It contains the following information:
 - Set up and operational details of each system.
 - Loading and unloading procedures.
 - Problems which could arise, their meaning reply and operation action.
 - Special checks and security measures.
 - Quick reference guides about operating a system in a short, concise format.

Training and Supporting Users

The type of training needed will vary by system type and user expertise. Types of training methods are:

- Traditional instructor-led classroom training
- E-learning/distance learning
- Blended learning (combination of instructor-led and e-learning)
- Software help components
- Electronic performance support system: component of a software package or an application in which training and educational information is embedded.

Training and Supporting Users

Computing supports for users has been provided in one of a few forums:

- i. Automating support:** Online support forums provide users access to information on new releases, bugs and tips for more effective users access to information on new releases, bugs and tips form more effective usage. Forums are offered over the internet or over company intranets.

- ii. Providing support through a help desk:** A help desk is an information systems department function and is staffed by IS personnel. The help desk is the first place users should call when they need assistance with an information system. The help desk staff members either deal with the users questions or refer the users to the most appropriate person.

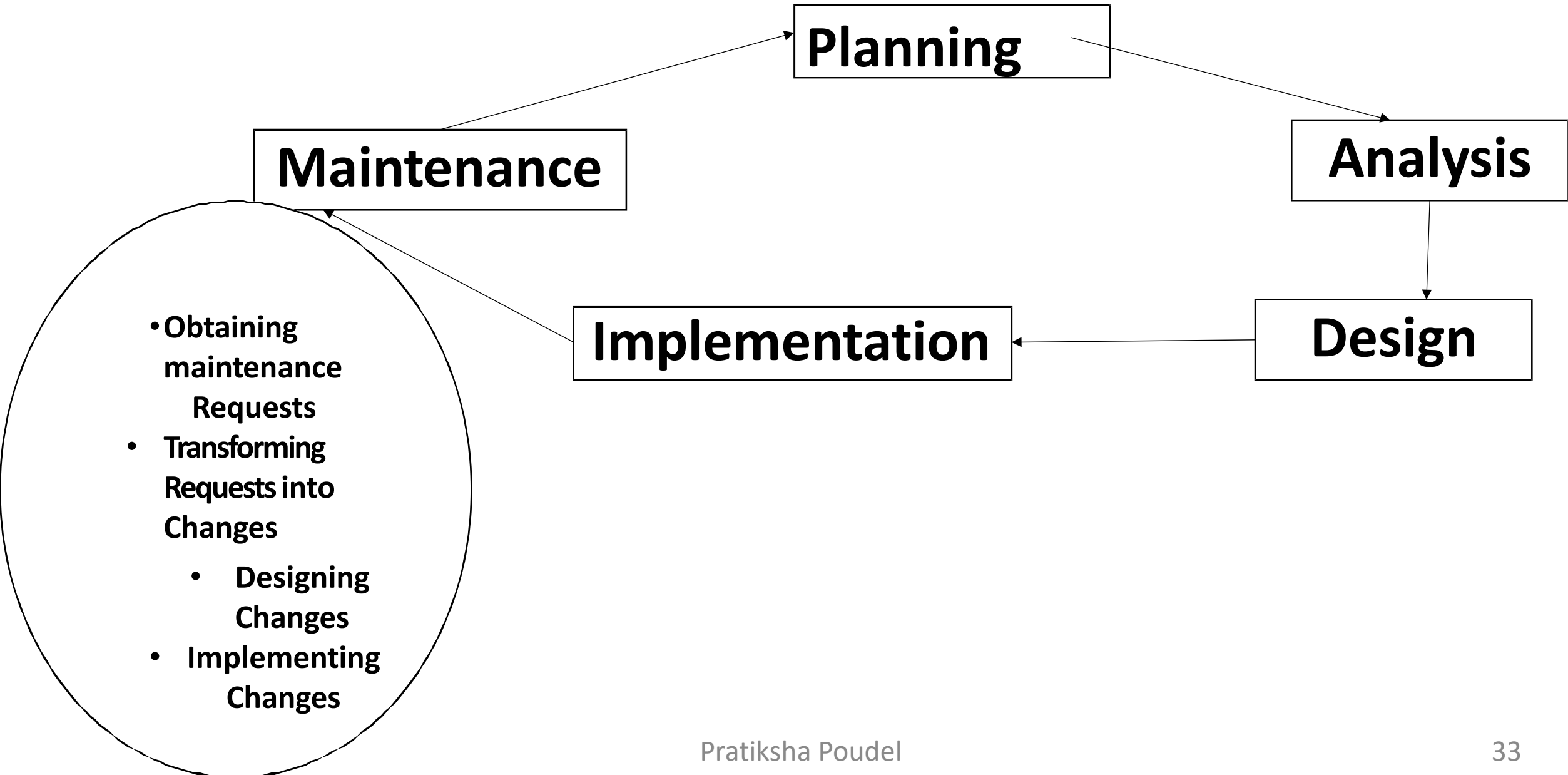
System Maintenance

- Software Maintenance is the process of modifying a software product after it has been delivered to the customer. The main purpose of software maintenance is to modify and update software applications after delivery to correct faults and to improve performance.
- Software Maintenance includes error correction, enhancements of capabilities, deletion of obsolete capabilities & optimization. As changes cannot be avoided, we should develop mechanism for evaluating, controlling & making modifications.
- After the product has been released, the maintenance phase keeps the s/w up to date with environment changes & changing user requirements.
- It consumes about 40-70% of the cost of the entire life cycle.
- Maintenance can only happen efficiently if the earlier phases are done properly.

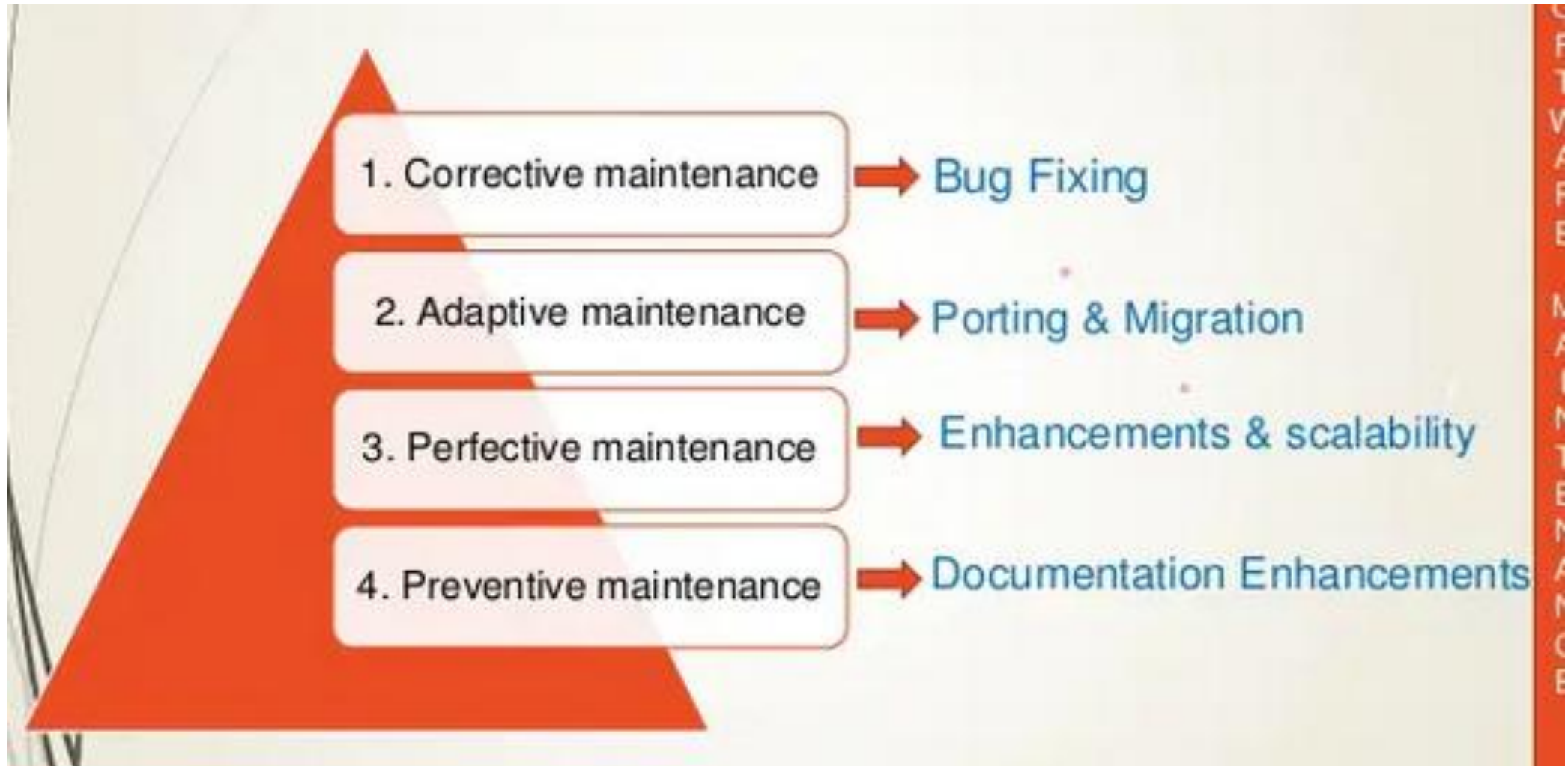
Need for Maintenance

- Correct errors
- Change in user requirement with time
- Changing hardware/software requirements
- To improve system efficiency
- To optimize the code to run faster
- To modify the components
- To reduce any unwanted side effects.

Analogy between SDLC and System Maintenance



Types of Maintainance



Corrective Maintenance:

- It refers to changes made to repair defects in the design, coding, or implementation of the system. This type of maintenance implies removing errors in a program, which might have crept in the system due to faulty design or wrong assumptions. Thus, in corrective maintenance, processing or performance failures are repaired.
- For example: If you had recently purchased a new home, corrective maintenance would involve repairs made to things that had never worked as designed, such as a faulty electrical outlet or a misaligned door.

2. Adaptive Maintenance

- This includes modifications and updates applied to keep the software product up-to date and tuned to the ever changing world of technology and business environment.
- This type of maintenance may become necessary because of organizational changes which may include:
 - ❖ Change in the organizational procedures
 - ❖ Change in organizational objectives, goals, policies etc.
 - ❖ Change in forms,
 - ❖ Change in information needs of managers.
 - ❖ Change in system controls and security needs etc.

Perfective Maintenance

- Perfective maintenance means adding new programs or modifying the existing programs to enhance the performance of the information system. This type of maintenance undertaken to respond to user's additional needs which may be due to the changes within or outside of the organization. Outside changes are primarily environmental changes, which may in the absence of system maintenance, render the information system ineffective and inefficient. These environmental changes include:
 - Changes in governmental policies, laws, etc
 - Economic and competitive conditions and
 - New technology.
- This includes modifications and updates done in order to keep the software usable over long period of time. It includes new features, new user requirements for refining the software and improve its reliability and performance.

Preventive Maintenance

- This type of maintenance includes modifications and updations to prevent future problems of the software. It goals to attend problems, which are not significant at this moment but may cause serious issues in future. .
- Includes anticipating future problems and to improve the maintainability using techniques like documenting, commenting or even re-implementing some part of documenting.
- More commonly known as Software Re-engineering.

The Cost of Maintenance

- The cost of maintenance represent a large proportion of the budget of most organization that use the software system. For some organizations, as much as 60 to 80 percent of their information systems budget is allocated to maintenance activities. These huge maintenance costs are due to the fact that many organizations have accumulated more and more older legacy systems that require more and more maintenance. More maintenance means more maintenance work for programmers. In addition, about one-third of the costs of establishing and keeping a presence on the Web go to programming maintenance.
- Maintenance cost as a proportion cost of development costs vary from one application domain to another. For business application systems, maintenance cost were broadly comparable with system development cost. For embedded real time systems, maintenance cost may be up to four times higher than development cost.
- So while developing a system, good software engineering techniques such as precise specification, use of object oriented development and configuration management are used that contribute to maintenance cost reduction.

The Cost of Maintenance

- Maintenance costs are usually greater than development costs.
- The costs arise from both technical and non-technical factors.
- A deployed system is expensive to change.
- High cost of breaking an already working system.
- Maintenance costs increase over time and as the system evolves. Reasons: Maintenance changes, and degrades the original system structure. Aging software results in high support costs.

Maintenance cost factors

There are two types of cost factors involved in software maintenance. These are

- **Non-Technical Factors**

1. **Application Domain**
2. **Staff Stability**
3. **Dependence on External Environment**
4. **Hardware Stability**

- **Technical Factors**

1. **Module Independence**
2. **Programming Language**
3. **Program Validation & Testing**
4. **Documentation**
5. **Configuration Management technique**

Maintenance cost factors

Non-Technical Factors

1. Application Domain

- If the application of the program is defined and well understood, the system requirements may be definitive and maintenance due to changing needs can be minimized.

2. Staff Stability

- It is simple for the original writer of a program to understand and change an application rather than some other person who must understand the program by the study of the reports and code listing.
- In practice, the feature of the programming profession is such that persons change jobs regularly. It is unusual for one user to develop and maintain an application throughout its useful life.

Maintenance cost factors

Non-Technical Factors

3. Dependence on External Environment

- If an application is dependent on its external environment, it must be modified as the working climate changes.

4. Hardware Stability

- If an application is designed to operate on a specific hardware configuration and that configuration does not change during the program's lifetime, no maintenance costs due to hardware changes will be incurred.

Maintenance cost factors

Technical Factors

1. Module Independence

- It should be possible to change one program unit of a system without affecting any other unit.

2. Programming Language

- Programs written in a high-level programming language are generally easier to understand than programs written in a low-level language.

3. Program Validation and Testing

- Generally, more the time and effort are spent on design validation and program testing, the fewer bugs in the program and, consequently, maintenance costs resulting from bugs correction are lower.

Maintenance cost factors

Technical Factors

4. Documentation

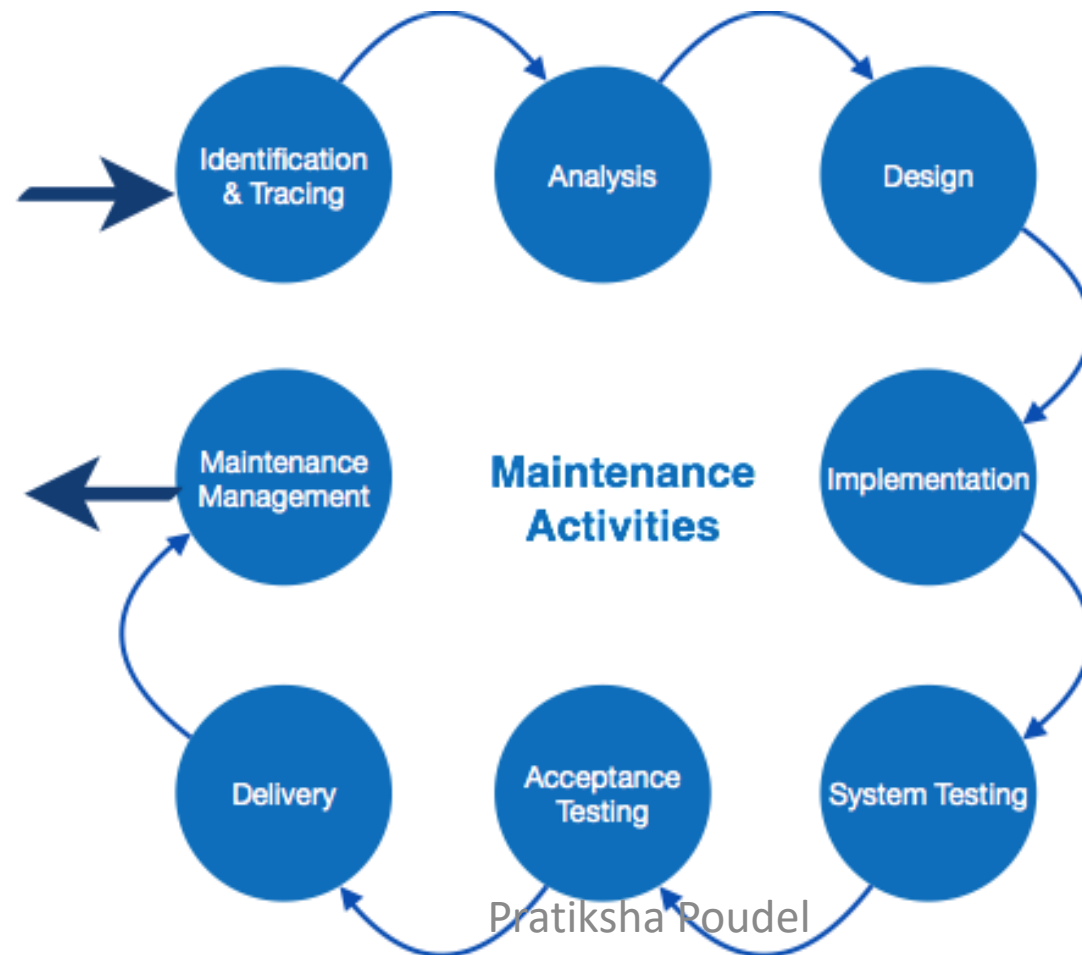
- If a program is supported by clear, complete yet concise documentation, the functions of understanding the application can be associatively straight-forward.
- Program maintenance costs tends to be less for well-reported systems than for the system supplied with inadequate or incomplete documentation.

5. Configuration Management Techniques

- One of the essential costs of maintenance is keeping track of all system documents and ensuring that these are kept consistent.
- Effective configuration management can help control these costs.

Maintenance Activities

IEEE provides a framework for sequential maintenance process activities. It can be used in iterative manner and can be extended so that customized items and processes can be included.



Maintenance Activities

Identification & Tracing - It involves activities for identification of requirement of modification or maintenance. It is generated by user or system may itself report via logs or error messages. Here, the maintenance type is classified also.

Analysis - The modification is analyzed for its impact on the system including safety and security implications. If probable impact is severe, alternative solution is looked for. A set of required modifications is then materialized into requirement specifications. The cost of modification/maintenance is analyzed and estimation is concluded.

Maintenance Activities

Design - New modules, which need to be replaced or modified, are designed against requirement specifications set in the previous stage. Test cases are created for validation and verification.

Implementation - The new modules are coded with the help of structured design created in the design step. Every programmer is expected to do unit testing in parallel.

System Testing - Integration testing is done among newly created modules. Integration testing is also carried out between new modules and the system. Finally the system is tested as a whole, following regressive testing procedures.

Maintenance Activities

Acceptance Testing - After testing the system internally, it is tested for acceptance with the help of users. If at this state, user complaints some issues they are addressed or noted to address in next iteration.

Delivery - After acceptance test, the system is deployed all over the organization either by small update package or fresh installation of the system. The final testing takes place at client end after the software is delivered.

Training facility is provided if required, in addition to the hard copy of user manual.

Maintenance management - Configuration management is an essential part of system maintenance. It is aided with version control tools to control versions, semi-version or patch management.

Maintenance Management

- Requires effective management, quality assurance and cost control
- To achieve these goals, companies use various strategies
- In addition, firms use version control and baselines to track system releases and analyze the system's life cycle.

Maintenance Management

- **The Maintenance Team**

- System administrator
- Systems analysts
 - Analysis
- Programmers
 - Applications programmer
 - Systems programmer (Server Configuration)
 - Database programmer
- Organizational issues
 - IT managers often divide systems analysts and programmers into two groups; one group performs new system development, and the other group handles maintenance.
 - Many analysts feel that maintenance work is less attractive than developing new systems.

Maintenance Management

Configuration Management

- Configuration Management (CM) is a process for controlling changes in system requirements during the development phases of the SDLC.
- As enterprise-wide information systems grow more complex, configuration management becomes critical.
- Also helps to organize and handle documentation.

System Performance Management

- Today companies use complex networks and client/server systems to support business needs.
- To ensure satisfactory support for business operations, the IT department must manage system faults and interruptions, measure system performance and workload and anticipate future needs.

System Performance Management

■ System Maintenance Tools

- Many CASE tools include system evaluation and maintenance features.
- In addition to CASE tools, spreadsheet and presentation software to calculate trends, perform what-if analyses, and create attractive charts and graphs to display the result.